Installation de NGINX sur les deux machines via la commande « dnf install nginx »

```
Résumé de la transaction
================================================================================
Installer  4 Paquets

Taille totale des téléchargements : 730 k
Taille des paquets installés : 1.9 M
Voulez-vous continuer ? [o/N] : o
Téléchargement des paquets :
(1/4): nginx-filesystem-1.26.3-1.el10.noarch.rpm
(2/4): nginx-1.26.3-1.el10.x86_64.rpm
(3/4): rocky-logos-httpd-100.4-7.el10.noarch.rpm
(4/4): nginx-core-1.26.3-1.el10.x86_64.rpm
--------------------------------------------------------------------------------
Total
Test de la transaction
La vérification de la transaction a réussi.
Lancement de la transaction de test
Transaction de test réussie.
Exécution de la transaction
  Préparation                  :
  Exécution du scriptlet: nginx-filesystem-2:1.26.3-1.el10.noarch
  Installation                 : nginx-filesystem-2:1.26.3-1.el10.noarch
  Installation                 : nginx-core-2:1.26.3-1.el10.x86_64
  Installation                 : rocky-logos-httpd-100.4-7.el10.noarch
  Installation                 : nginx-2:1.26.3-1.el10.x86_64
  Exécution du scriptlet: nginx-2:1.26.3-1.el10.x86_64
[  586.159048] systemd-rc-local-generator[33316]: /etc/rc.d/rc.local is not marked executable,

Installé:
  nginx-2:1.26.3-1.el10.x86_64     nginx-core-2:1.26.3-1.el10.x86_64     nginx-filesystem-2:1.2
```

On ouvre les ports pour les service http et https et on relance le pare-feu:

```
[root@localhost ~]# firewall-cmd --add-service=http --permanent
success
[root@localhost ~]# firewall-cmd --add-service=https --permanent
success
```

On vérifie que les services sont bien ouverts via la commande « firewall-cmd –list-service »

```
[root@localhost ~]# firewall-cmd --list-service
cockpit dhcpv6-client http https ssh
```

On va créer la page web par défaut de chaque serveur web en commencant par créer le répertoire où sera accueilli les pages

```
[root@localhost ~]# mkdir -p /var/www/web/
```

Puis on crée la page d'accueil avec nano

```
[root@localhost web]# nano index.html
```

```
  GNU nano 8.1                    index.html                    Modifié
<p>Bonjour je suis le serveur "SRV WEB 1"</p>

^G Aide      ^O Écrire    ^F Chercher  ^K Couper    ^T Exécuter  ^C Emplacement
^X Quitter   ^R Lire fich.^\ Remplacer  ^U Coller    ^J Justifier ^/ Aller ligne
```

On se rend dans le repertoire « etc/nginx/conf.d/ » et on crée le fichier « web.conf » que l'on vient remplir :

```
[root@localhost web]# cd /etc/nginx/conf.d/
[root@localhost conf.d]# nano web.conf
```

```
  GNU nano 8.1                    web.conf
server {
        listen 80;
        listen [::]:80;

        root /var/www/web/ ;
        index index.html index.htm index.nginx-debian.html sign-up.php;

        server_name web.fr ;
  location ~* \.php$ {
    fastcgi_pass unix:/run/php-fpm/www.sock;
    include         fastcgi_params;
    fastcgi_param   SCRIPT_FILENAME    $document_root$fastcgi_script_name;
    fastcgi_param   SCRIPT_NAME        $fastcgi_script_name;
}
        access_log /var/log/nginx/access_web.log;
        error_log /var/log/nginx/error_web.log;
        location / {
                try_files $uri $uri/ =404;
        }
}
                                [ Lecture de 20 lignes ]
```

On relance le service nginx car on a modifié le fichier de config et on active le service :

```
[root@localhost conf.d]# systemctl restart nginx
[root@localhost conf.d]# systemctl enable nginx
Created symlink '/etc/systemd/system/multi-user.target.wants/nginx.service' → '/
usr/lib/systemd/system/nginx.service'.
```

On désactive selinux via la commande « setenforce 0 » :

```
[root@localhost ~]# setenforce 0
[root@localhost ~]#
```

On crée le repertoire /etc/ssl/private

```
[root@localhost ~]# mkdir /etc/ssl/private
[root@localhost ~]#
```

On limite les accès sur ce répertoire :

```
[root@localhost ~]# chmod 700 /etc/ssl/private
```

On génére les certificats avec la commande « openssl req -x509 -nodes -days 365 -newkey rsa:2048 -keyout /etc/ssl/private/nginx-selsigned.key -out /etc/ssl/certs/nginx-selfsigned.crt »

```
[root@localhost ~]# openssl req -x509 -nodes -days 365 -newkey rsa:2048 -keyout
/etc/ssl/private/nginx-selsigned.key -out /etc/ssl/certs/nginx-selfsigned.crt
...+...+...+.....+...++++++++++++++++++++++++++++++++++++++++*.......+......+...
+.....+.+...+.....+...+...+.......+........+....+..+.+..........
....+++++++++++++++++++++++++++++++++++++*...+....+........+.......+..........
.....+...+..+.........+.....+.+.............+......+.+.....+......++++++
...+.....+......+...+......+.........+............+...+......+.+..+............+
....+......+......+..+.+..+.............+.....+......++++++++++++++++++++++++++
++++++++++++*...+.............+.........+.+......+..++++++++++++++++++++++++++
++++++++++++*....+....+..+...+......+.+..+.+......+...+......+...............+..
....+.....+...+......+.+.....+.+..+.................+...+....+...+........+....
..+.+.........+...........+.......+......+......+......+..+.+.....+.+......+....
.+...............+.....................++++++
-----
You are about to be asked to enter information that will be incorporated
into your certificate request.
What you are about to enter is what is called a Distinguished Name or a DN.
There are quite a few fields but you can leave some blank
For some fields there will be a default value,
If you enter '.', the field will be left blank.
-----
Country Name (2 letter code) [XX]:FR
State or Province Name (full name) []:Nord
Locality Name (eg, city) [Default City]:Lille
Organization Name (eg, company) [Default Company Ltd]:Gaston Berger
Organizational Unit Name (eg, section) []:SIO
Common Name (eg, your name or your server's hostname) []:SRV WEB 1
Email Address []:
```

On génére un groupe diffie-hellman fort qui permet la négociation de PFS :

```
[root@localhost ~]# openssl dhparam -out /etc/ssl/certs/dhparam.pem 2048
Generating DH parameters, 2048 bit long safe prime
.....................................................................................................+....
.....................................................................................................
...............+.....................................................................................+...............
.....................................................................................................
....+....................+.........................................................................................
.....................................................................................................
...........................+.......................................+.........................................
................+...............................+.....................................................
.........................................+.......................................................+.............
.....................................................................................................
.....................................................................................................
....+...............+.................................................................................
.....................................................................................................
.....................................................................................................+.............
.....................................................................................................
.....................................................................................................
.......................................................++*++*++*
```

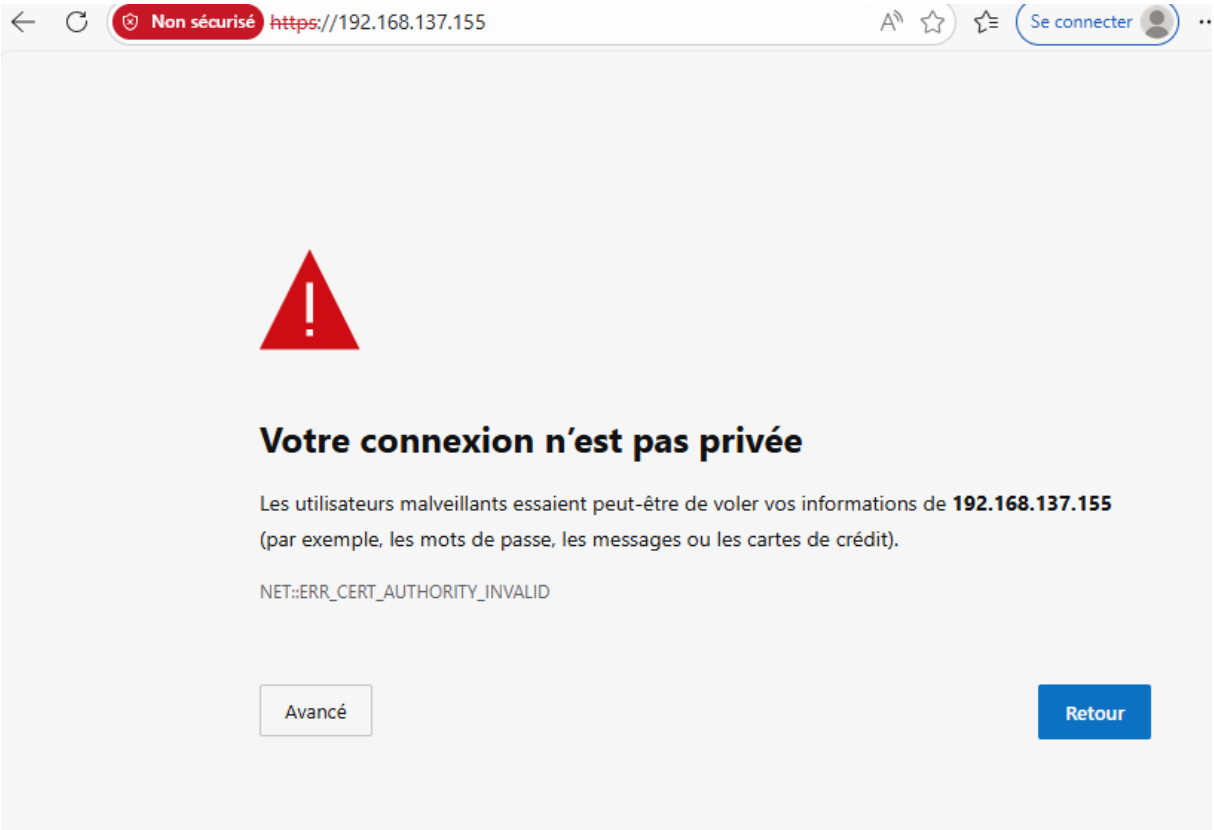
On teste nos fichiers avec « nginx -t »

```
[root@localhost ~]# nginx -t
nginx: [warn] the "listen ... http2" directive is deprecated, use the "http2" directive instead in /etc/nginx/conf.d/
web.conf:23
nginx: [warn] the "listen ... http2" directive is deprecated, use the "http2" directive instead in /etc/nginx/conf.d/
web.conf:24
nginx: the configuration file /etc/nginx/nginx.conf syntax is ok
```

On relance le service nginx car on a modifié le fichier de config nginx

```
[root@localhost private]# systemctl restart nginx
```

Quand on essaye d'accéder au site on a un avertissement et le site apparait comme étant non sécurisé



On installe le package haproxy sur les serveurs WEB

```
[root@localhost private]# dnf install haproxy
Dernière vérification de l'expiration des métadonnées effectuée il y a 0:14:30 le lun. 23 mars 2026 10:49:09.
Dépendances résolues.
================================================================================================================
 Paquet                  Architecture            Version                         Dépôt                     Taille
================================================================================================================
Installation:
 haproxy                 x86_64                  3.0.5-4.el10_1.1                appstream                  2.6 M

Résumé de la transaction
================================================================================================================
Installer  1 Paquet

Taille totale des téléchargements : 2.6 M
Taille des paquets installés : 8.0 M
Voulez-vous continuer ? [o/N] : o
Téléchargement des paquets :
haproxy-3.0.5-4.el10_1.1.x86_64.rpm                                          20 MB/s | 2.6 MB     00:00
----------------------------------------------------------------------------------------------------------------
Total                                                                        7.3 MB/s | 2.6 MB     00:00
Test de la transaction
La vérification de la transaction a réussi.
Lancement de la transaction de test
Transaction de test réussie.
Exécution de la transaction
  Préparation                  :                                                                              1/1
  Exécution du scriptlet: haproxy-3.0.5-4.el10_1.1.x86_64                                                     1/1
  Installation                 : haproxy-3.0.5-4.el10_1.1.x86_64                                              1/1
  Exécution du scriptlet: haproxy-3.0.5-4.el10_1.1.x86_64                                                     1/1

Installé:
  haproxy-3.0.5-4.el10_1.1.x86_64

Terminé !
```

On modifie le fichier de config « haproxy.cfg » se trouvant dans « /etc/haproxy/ » et on y ajoute nos serveurs

```
#---------------------------------------------------------------------
# round robin balancing between the various backends
#---------------------------------------------------------------------
backend app
    balance     roundrobin
    server  web01 192.168.137.155:443 ssl verify none check
    server  web02 192.168.137.154:443 ssl veirfy none check
```

On active Rsyslog en déconnectant les deux lignes surlignés ci-dessous dans le fichier « /etc/rsyslog.conf » :

```
# Provides UDP syslog reception
# for parameters see http://www.rsyslog.com/doc/imudp.html
module(load="imudp") # needs to be done just once
input(type="imudp" port="514")
```

On attribue une policy a SElinux

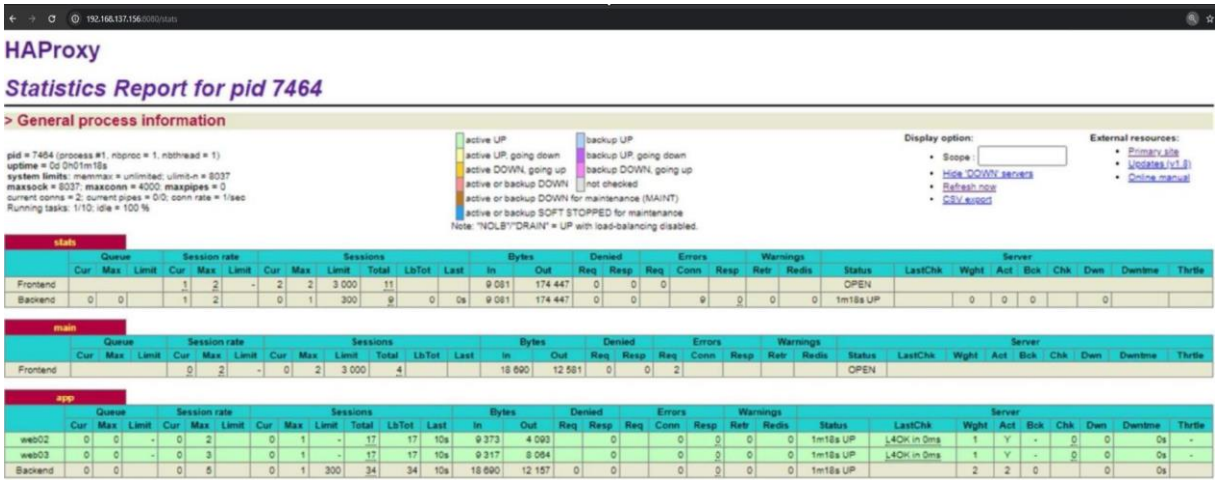
```
[root@localhost ~]# setsebool -P haproxy_connect_any 1
```

On démarre les services haproxy et rsyslog

```
[root@localhost ~]# systemctl start haproxy
[root@localhost ~]# systemctl start rsyslog
```

A. Depuis le serveur HA Proxy

On vérifie dans un navigateur web l'accès à la page d'administration en se rendant sur http://192.168.137.156:8080/stats :



On vérifie également que le service haproxy est actif depuis la console :

```
[root@localhost ~]# systemctl status haproxy
haproxy.service - HAProxy Load Balancer
   Loaded: loaded (/usr/lib/systemd/system/haproxy.service; enabled
   Active: active (running) since Mon 2024-11-18 09:12:04 CET; 12 da
 Main PID: 7464 (haproxy)
```

B. Sécurisation

On se place dans le répertoire /etc/pki/tls/certs puis on génère un certificat auto-signé pour HAProxy :

```
[root@localhost ~]# cd /etc/pki/tls/certs
[root@localhost certs]# openssl req -x509 -nodes -newkey rsa:2048 -k
Generating a RSA private key
..............................+++++
..............................+++++
writing new private key to '/etc/pki/tls/certs/haproxy.pem'
-----
You are about to be asked to enter information...
Country Name (2 letter code) [XX]:FR
State or Province Name (full name) []:
Common Name (eg, your server's hostname) []:
Email Address []:
```

Une fois le certificat généré, on modifie /etc/haproxy/haproxy.cfg pour ajouter les paramètres SSL dans les sections global et frontend :

```
[root@localhost ~]# nano /etc/haproxy/haproxy.cfg
# Dans la section global :
    maxsslconn 256
    tune.ssl.default-dh-param 2048
# Dans la section frontend :
    bind *:443 ssl crt /etc/pki/tls/certs/haproxy.pem
```

On redémarre le service pour appliquer la configuration :

```
[root@localhost ~]# systemctl restart haproxy
[root@localhost ~]# systemctl status haproxy
haproxy.service - HAProxy Load Balancer
   Active: active (running)
```

C. Critères de réussite

On vérifie le round-robin : en accédant plusieurs fois via l'IP du HAProxy (192.168.137.156), les requêtes alternent entre WEB01 et WEB02 :

```
[root@localhost ~]# curl -k https://192.168.137.156
Bonjour je suis le serveur WEB01
[root@localhost ~]# curl -k https://192.168.137.156
Bonjour je suis le serveur WEB02
[root@localhost ~]# curl -k https://192.168.137.156
Bonjour je suis le serveur WEB01
```

On teste la bascule en arrêtant nginx sur WEB01. Tout le trafic est redirigé automatiquement vers WEB02 :

```
[root@localhost ~]# systemctl stop nginx
```

```
[root@localhost ~]# curl -k https://192.168.137.156
Bonjour je suis le serveur WEB02
[root@localhost ~]# curl -k https://192.168.137.156
Bonjour je suis le serveur WEB02
```